JobSheet - Week 7 – Java Stream API

Nama: Muhammad Fawwaz Muzakki

NIM: 251524056

Kelas: 1B

Repo GitHub: <Tuliskan Tautan Repo GitHub>

Instruksi Pengerjaan:

1. Kerjakan 3 soal di bawah ini dengan melengkapi setiap kolom jawaban yang disediakan pada jobsheet ini.
2. Jawaban setiap soal mencakup source code, screenshot hasil dari program yang ditampilkan full screen termasuk taskbar (tambahkan beberapa screenshot jika diperlukan), penjelasan permasalahan dan solusi yang dihadapi, nama teman yang membantu memecahkan masalah (opsional).
3. Dikumpulkan pada Assignment Classroom sesuai dengan deadline yang tertera pada assignment tersebut.
4. Format penamaan file jobsheet: W7_P_<Kelas 1X>_<3 Digit_NIM_Terakhir>.docx/pdf. Contoh: W7_P_1B_001.docx/pdf.
5. Submit semua jawaban dalam bentuk file java pada repository GitHub masing-masing.

No. 1 Transfer Fulus

Soal Praktikum

Instruksi Pengerjaan:
a. **Analisis Masalah**: Perhatikan source code di bawah ini, masalah apa yang Anda temukan? Berikan penjelasannya pada setiap bagian source code dengan menyeluruh berdasarkan materi yang telah diperoleh pada kelas teori!
b. **Solusi Terhadap Masalah**: Tuliskan solusi berupa source code dan penjelasan solusi terhadap masalah tersebut! Berikan penjelasannya pada setiap bagian source code!
 	
Source Code:
class Account {
  int balance = 150;
}
 	
public class TransferFulus {

```
  public static void main(String[] args) throws InterruptedException {
    Account acc1 = new Account();
    Account acc2 = new Account();

    // Thread 1: Menjumlahkan/ transfer fulus dari acc1 ke acc2
    Thread t1 = new Thread(() -> {
      synchronized (acc1) { // Berikan komentar apa yang dilakukan pada blok ini?
        System.out.println("Berikan pesan yang di cetak sebagai penanda thread ini
sedang melakukan apa?");
        try { Thread.sleep(100); } catch (Exception e) {} // Simulasi dengan
memberikan jeda. Mengapa diperlukan Exception?

        synchronized (acc2) { // Berikan komentar apa yang dilakukan pada blok ini?
          System.out.println("Berikan pesan yang di cetak sebagai penanda thread ini
sedang melakukan apa?");
          acc2.balance += acc1.balance;
        }
      }
    });

    // Thread 2: Menjumlahkan/ transfer fulus dari acc2 ke acc1
    Thread t2 = new Thread(() -> {
      synchronized (acc2) { // Berikan komentar apa yang dilakukan pada blok ini?
        System.out.println("Berikan pesan yang di cetak sebagai penanda thread ini
sedang melakukan apa?");
        try { Thread.sleep(100); } catch (Exception e) {}

        synchronized (acc1) { // Berikan komentar apa yang dilakukan pada blok ini?
          System.out.println("Berikan pesan yang di cetak sebagai penanda thread ini
sedang melakukan apa?");
          acc1.balance += acc2.balance;
        }
      }
    });

    t1.start();
    t2.start();

    t1.join();
    t2.join();

    System.out.println("--- HASIL AKHIR ---");
    System.out.println("Saldo Akhir acc1: " + acc1.balance);
                System.out.println("Saldo Akhir acc2: " + acc2.balance);
  }
}
```

Source Code

```java
class Account {
    int balance = 150;
}

public class TransferFulus {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) throws InterruptedException {
        Account acc1 = new Account();
        Account acc2 = new Account();

        // Thread 1: Menjumlahkan/ transfer fulus dari acc1 ke acc2
        Thread t1 = new Thread(() -> {
            synchronized (acc1) { // Berikan komentar apa yang dilakukan pada blok ini?
                System.out.println(x: "Berikan pesan yang di cetak sebagai penanda thread ini sedang melakukan apa?");
                try {
                    Thread.sleep(millis: 1000);    Thread.sleep called in synchronized context
                } catch (Exception e) {    Can be replaced with multicatch or several catch clauses catching specific exceptions
                } // Simulasi dengan memberikan jeda. Mengapa diperlukan Exception?

                synchronized (acc2) { // Berikan komentar apa yang dilakukan pada blok ini?
                    System.out.println(x: "Berikan pesan yang di cetak sebagai penanda thread ini sedang melakukan apa?");
                    acc2.balance += acc1.balance;
                }
            }
        });

        // Thread 2: Menjumlahkan/ transfer fulus dari acc2 ke acc1
        Thread t2 = new Thread(() -> {
            synchronized (acc1) { // Berikan komentar apa yang dilakukan pada blok ini?
                System.out.println(x: "Berikan pesan yang di cetak sebagai penanda thread ini sedang melakukan apa?");
                try {
                    Thread.sleep(millis: 1000);    Thread.sleep called in synchronized context
                } catch (Exception e) {    Can be replaced with multicatch or several catch clauses catching specific exceptions
                }

                synchronized (acc2) { // Berikan komentar apa yang dilakukan pada blok ini?
                    System.out.println(x: "Berikan pesan yang di cetak sebagai penanda thread ini sedang melakukan apa?");
                    acc1.balance += acc2.balance;

                }
            }
        });

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println(x: "--- HASIL AKHIR ---");
        System.out.println("Saldo Akhir acc1: " + acc1.balance);
        System.out.println("Saldo Akhir acc2: " + acc2.balance);
    }
}
```

Screenshot Hasil

```
Berikan pesan yang di cetak sebagai penanda thread ini sedang melakukan apa?
Berikan pesan yang di cetak sebagai penanda thread ini sedang melakukan apa?
Berikan pesan yang di cetak sebagai penanda thread ini sedang melakukan apa?
Berikan pesan yang di cetak sebagai penanda thread ini sedang melakukan apa?
--- HASIL AKHIR ---
Saldo Akhir acc1: 450
Saldo Akhir acc2: 300
```

Penjelasan

Dalam penggunaan synchronized harus berurut karena sebelumnya kode ini mengalami masalah yakni Deadlock – buntu, Ketika thread 1 menutup thread 2 menutup saat Langkah kedua dua-duamya saling menutup jadi tidak ada program yang berjalan.

Maka dari itu diperlukannya synchronized secara terurut agar tidak terjadi Deadlock

Nama Teman Hal yang Dibantu (Opsional)

No. 2 Resto Ayam

Soal Praktikum

Instruksi Pengerjaan:

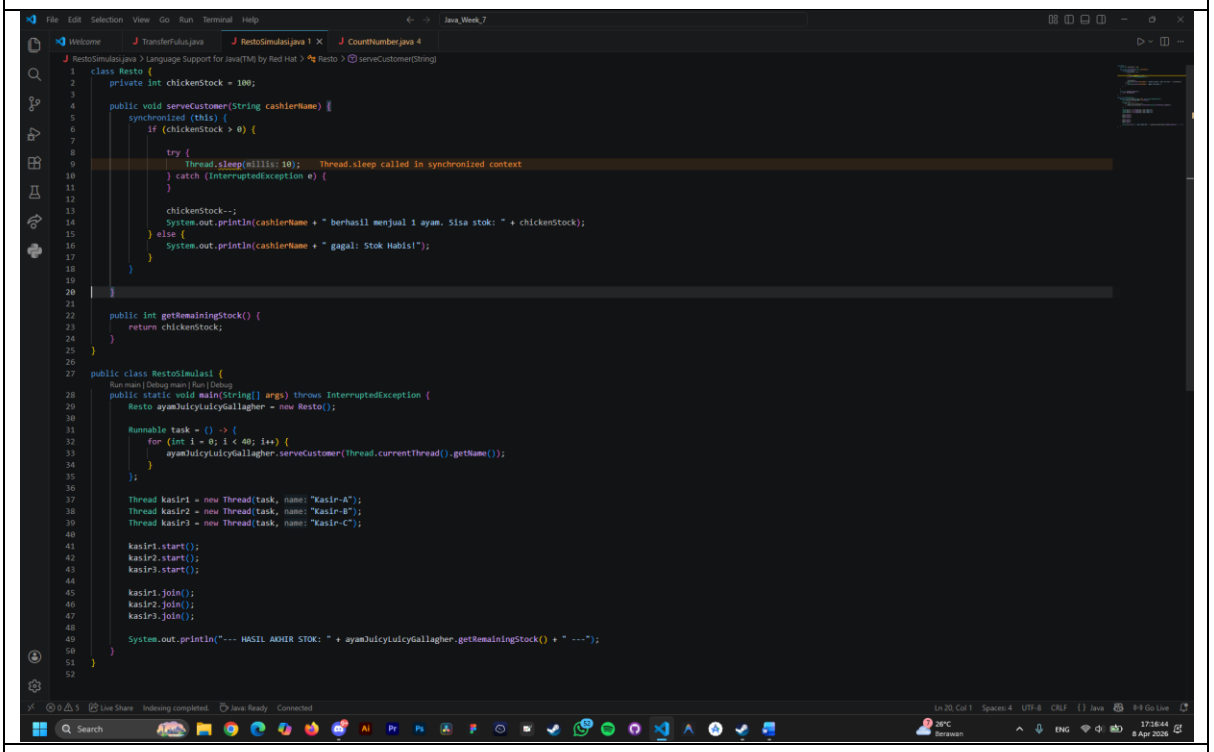
a. **Analisis Masalah**: Perhatikan source code di bawah ini, masalah apa yang Anda temukan? Berikan penjelasannya pada setiap bagian source code dengan menyeluruh berdasarkan materi yang telah diperoleh pada kelas teori!
b. **Solusi Terhadap Masalah**: Tuliskan solusi berupa source code dan penjelasan solusi terhadap masalah tersebut! Berikan penjelasannya pada setiap bagian source code!

Source Code:

```
class Resto {
  private int chickenStock = 100;

  public void serveCustomer(String cashierName) {
    if (chickenStock > 0) {
      try { Thread.sleep(10); } catch (InterruptedException e) {}

      chickenStock--;
      System.out.println(cashierName + " berhasil menjual 1 ayam. Sisa stok: " +
chickenStock);
    } else {
      System.out.println(cashierName + " gagal: Stok Habis!");
    }
  }

  public int getRemainingStock() {
    return chickenStock;
  }
}

public class RestoSimulasi {
  public static void main(String[] args) throws InterruptedException {
    Resto ayamJuicyLuicyGallagher = new Resto();

    Runnable task = () -> {
      for (int i = 0; i < 40; i++) {

ayamJuicyLuicyGallagher.serveCustomer(Thread.currentThread().getName());
      }
    };

    Thread kasir1 = new Thread(task, "Kasir-A");
    Thread kasir2 = new Thread(task, "Kasir-B");
    Thread kasir3 = new Thread(task, "Kasir-C");
```

```
        kasir1.start();
        kasir2.start();
        kasir3.start();

        kasir1.join();
        kasir2.join();
        kasir3.join();

        System.out.println("--- HASIL AKHIR STOK: " +
ayamJuicyLuicyGallagher.getRemainingStock() + " ---");
    }
}
```

Source Code



Screenshot Hasil

```
Kasir-C berhasil menjual 1 ayam. Sisa stok: 57
Kasir-C berhasil menjual 1 ayam. Sisa stok: 56
Kasir-C berhasil menjual 1 ayam. Sisa stok: 55
Kasir-C berhasil menjual 1 ayam. Sisa stok: 54
Kasir-C berhasil menjual 1 ayam. Sisa stok: 53
Kasir-C berhasil menjual 1 ayam. Sisa stok: 52
Kasir-C berhasil menjual 1 ayam. Sisa stok: 51
Kasir-C berhasil menjual 1 ayam. Sisa stok: 50
Kasir-C berhasil menjual 1 ayam. Sisa stok: 49
Kasir-C berhasil menjual 1 ayam. Sisa stok: 48
Kasir-C berhasil menjual 1 ayam. Sisa stok: 47
Kasir-C berhasil menjual 1 ayam. Sisa stok: 46
Kasir-C berhasil menjual 1 ayam. Sisa stok: 45
Kasir-C berhasil menjual 1 ayam. Sisa stok: 44
Kasir-A berhasil menjual 1 ayam. Sisa stok: 43
Kasir-A berhasil menjual 1 ayam. Sisa stok: 42
Kasir-A berhasil menjual 1 ayam. Sisa stok: 41
Kasir-A berhasil menjual 1 ayam. Sisa stok: 40
Kasir-A berhasil menjual 1 ayam. Sisa stok: 39
Kasir-A berhasil menjual 1 ayam. Sisa stok: 38
Kasir-A berhasil menjual 1 ayam. Sisa stok: 37
Kasir-A berhasil menjual 1 ayam. Sisa stok: 36
Kasir-A berhasil menjual 1 ayam. Sisa stok: 35
Kasir-A berhasil menjual 1 ayam. Sisa stok: 34
Kasir-A berhasil menjual 1 ayam. Sisa stok: 33
Kasir-A berhasil menjual 1 ayam. Sisa stok: 32
Kasir-A berhasil menjual 1 ayam. Sisa stok: 31
Kasir-A berhasil menjual 1 ayam. Sisa stok: 30
Kasir-A berhasil menjual 1 ayam. Sisa stok: 29
Kasir-A berhasil menjual 1 ayam. Sisa stok: 28
Kasir-A berhasil menjual 1 ayam. Sisa stok: 27
Kasir-A berhasil menjual 1 ayam. Sisa stok: 26
Kasir-A berhasil menjual 1 ayam. Sisa stok: 25
Kasir-A berhasil menjual 1 ayam. Sisa stok: 24
Kasir-A berhasil menjual 1 ayam. Sisa stok: 23
Kasir-A berhasil menjual 1 ayam. Sisa stok: 22
Kasir-A berhasil menjual 1 ayam. Sisa stok: 21
Kasir-A berhasil menjual 1 ayam. Sisa stok: 20
Kasir-A berhasil menjual 1 ayam. Sisa stok: 19
Kasir-A berhasil menjual 1 ayam. Sisa stok: 18
Kasir-A berhasil menjual 1 ayam. Sisa stok: 17
```

```
Kasir-A berhasil menjual 1 ayam. Sisa stok: 17
Kasir-A berhasil menjual 1 ayam. Sisa stok: 16
Kasir-A berhasil menjual 1 ayam. Sisa stok: 15
Kasir-A berhasil menjual 1 ayam. Sisa stok: 14
Kasir-A berhasil menjual 1 ayam. Sisa stok: 13
Kasir-B berhasil menjual 1 ayam. Sisa stok: 12
Kasir-B berhasil menjual 1 ayam. Sisa stok: 11
Kasir-C berhasil menjual 1 ayam. Sisa stok: 10
Kasir-C berhasil menjual 1 ayam. Sisa stok: 9
Kasir-C berhasil menjual 1 ayam. Sisa stok: 8
Kasir-C berhasil menjual 1 ayam. Sisa stok: 7
Kasir-C berhasil menjual 1 ayam. Sisa stok: 6
Kasir-C berhasil menjual 1 ayam. Sisa stok: 5
Kasir-C berhasil menjual 1 ayam. Sisa stok: 4
Kasir-C berhasil menjual 1 ayam. Sisa stok: 3
Kasir-C berhasil menjual 1 ayam. Sisa stok: 2
Kasir-C berhasil menjual 1 ayam. Sisa stok: 1
Kasir-C berhasil menjual 1 ayam. Sisa stok: 0
Kasir-C gagal: Stok Habis!
Kasir-C gagal: Stok Habis!
Kasir-C gagal: Stok Habis!
Kasir-C gagal: Stok Habis!
Kasir-B gagal: Stok Habis!
Kasir-B gagal: Stok Habis!
Kasir-B gagal: Stok Habis!
Kasir-B gagal: Stok Habis!
Kasir-B gagal: Stok Habis!
Kasir-B gagal: Stok Habis!
Kasir-B gagal: Stok Habis!
Kasir-B gagal: Stok Habis!
Kasir-B gagal: Stok Habis!
Kasir-B gagal: Stok Habis!
Kasir-B gagal: Stok Habis!
Kasir-B gagal: Stok Habis!
Kasir-B gagal: Stok Habis!
Kasir-B gagal: Stok Habis!
Kasir-B gagal: Stok Habis!
--- HASIL AKHIR STOK: 0 ---
```

```
PS C:\Fawwaz\Tugas Kuliah\Tekprom\TekProm(pr)\Java_Week_7>  &
aceStorage\76670915e580c1b9877abf9ef0754c87\redhat.java\jdt_w
Kasir-A berhasil menjual 1 ayam. Sisa stok: 99
Kasir-A berhasil menjual 1 ayam. Sisa stok: 98
Kasir-A berhasil menjual 1 ayam. Sisa stok: 97
Kasir-A berhasil menjual 1 ayam. Sisa stok: 96
Kasir-A berhasil menjual 1 ayam. Sisa stok: 95
Kasir-A berhasil menjual 1 ayam. Sisa stok: 94
Kasir-A berhasil menjual 1 ayam. Sisa stok: 93
Kasir-A berhasil menjual 1 ayam. Sisa stok: 92
Kasir-A berhasil menjual 1 ayam. Sisa stok: 91
Kasir-B berhasil menjual 1 ayam. Sisa stok: 90
Kasir-B berhasil menjual 1 ayam. Sisa stok: 89
Kasir-B berhasil menjual 1 ayam. Sisa stok: 88
Kasir-B berhasil menjual 1 ayam. Sisa stok: 87
Kasir-B berhasil menjual 1 ayam. Sisa stok: 86
Kasir-B berhasil menjual 1 ayam. Sisa stok: 85
Kasir-B berhasil menjual 1 ayam. Sisa stok: 84
Kasir-B berhasil menjual 1 ayam. Sisa stok: 83
Kasir-B berhasil menjual 1 ayam. Sisa stok: 82
Kasir-B berhasil menjual 1 ayam. Sisa stok: 81
Kasir-B berhasil menjual 1 ayam. Sisa stok: 80
Kasir-B berhasil menjual 1 ayam. Sisa stok: 79
Kasir-B berhasil menjual 1 ayam. Sisa stok: 78
Kasir-B berhasil menjual 1 ayam. Sisa stok: 77
Kasir-B berhasil menjual 1 ayam. Sisa stok: 76
Kasir-B berhasil menjual 1 ayam. Sisa stok: 75
Kasir-B berhasil menjual 1 ayam. Sisa stok: 74
Kasir-B berhasil menjual 1 ayam. Sisa stok: 73
Kasir-B berhasil menjual 1 ayam. Sisa stok: 72
Kasir-B berhasil menjual 1 ayam. Sisa stok: 71
Kasir-B berhasil menjual 1 ayam. Sisa stok: 70
Kasir-B berhasil menjual 1 ayam. Sisa stok: 69
Kasir-C berhasil menjual 1 ayam. Sisa stok: 68
Kasir-C berhasil menjual 1 ayam. Sisa stok: 67
Kasir-C berhasil menjual 1 ayam. Sisa stok: 66
Kasir-C berhasil menjual 1 ayam. Sisa stok: 65
Kasir-C berhasil menjual 1 ayam. Sisa stok: 64
Kasir-C berhasil menjual 1 ayam. Sisa stok: 63
Kasir-C berhasil menjual 1 ayam. Sisa stok: 62
Kasir-C berhasil menjual 1 ayam. Sisa stok: 61
Kasir-C berhasil menjual 1 ayam. Sisa stok: 60
Kasir-C berhasil menjual 1 ayam. Sisa stok: 59
Kasir-C berhasil menjual 1 ayam. Sisa stok: 58
```

Penjelasan

Dalam penggunaan thread diperlukannya synchronized agar tidak terjadinya race condition – saling bertabrakan.
Dalam program ini saya menggunakan synchronized agar tidak terjadi race condition yang Dimana nanti data akan bertabrakan, karena thread yang saya gunakan Adalah 3, maka ia akan berulang sebanyak 3x40 = 120 sampai akhirnya menjadi 0/kosong.

Nama Teman Hal yang Dibantu (Opsional)

No. 3 Penjumlahan Paralel

Soal Praktikum
Instruksi Pengerjaan: Buatlah suatu program yang bertujuan untuk melakukan penjumlahan secara paralel dengan ketentuan sebagai berikut: - Menerima Input: (1) Jumlah Thread; (2) Angka Akhir - Contoh: Menjumlahkan angka dari 1 sampai "Angka Akhir". Jika "Jumlah Thread" = 4, "Angka Akhir" = 1000, maka: - Thread 1: Menjumlahkan 1 - 250 - Thread 2: Menjumlahkan 251 - 500 - Thread 3: Menjumlahkan 501 - 750 - Thread 4: Menjumlahkan 751 - 1000 - Hasil akhir diperoleh dari hasil parsial (dari masing-masing thread) yang dijumlahkan menjadi total akhir. - Perhatikan Thread Safety, Mekanisme Pembagian Tugas (Divide and Conquer), dan Syncronization agar hasil akhir sesuai! - Pada program, tampilkan output: - Tugas apa yang dikerjakan oleh masing-masing thread - Hasil parsial dari masing-masing thread - Hasil akhir yang merupakan penjumlahan dari semua hasil parsial - Berikan penjelasan yang lengkap untuk setiap bagian pentingnya!
Source Code

```java
import java.util.Scanner;

class Number implements Runnable {
    private int start, end;    Field start can be final
    private long partialSum = 0;
    private String threadName;    Field threadName can be final

    public Number(int start, int end, String name) {
        this.start = start;
        this.end = end;
        this.threadName = name;
    }

    @Override
    public void run() {
        System.out.println(threadName + " sedang menjumlahkan: " + start + " sampai " + end);
        for (int i = 0; i <= end; i++) {
            partialSum += i;
        }

    }

    public long getPartialSum() {
        return partialSum;
    }
}

public class CountNumber {
    Run main | Debug main | Run | Debug
    public static void main(String[] args) {
            Scanner scanner = new Scanner (System.in);    Convert to try-with-resources

            System.out.println(x: "Masukkan jumlah thread: ");
            int numThreads = scanner.nextInt();
            System.out.println(x: "\nMasukkan angka akhir: ");
            int lastNumber = scanner.nextInt();

            Thread[] threads = new Thread[numThreads];
            Number[] numbers = new Number[numThreads];

            int gap = lastNumber / numThreads;
            int currentStart = 1;

            for(int i = 0; i < numThreads; i++){
                int currentEnd = (i == numThreads - 1) ? lastNumber : (currentStart + gap - 1);

                numbers[i] = new Number(currentStart, currentEnd, " Thread " + (i+1));

                threads[i] = new Thread(numbers[i]);

                threads[i].start();

                currentStart = currentEnd + 1;
            }

            long totalsum = 0;
            try {
                for(int i = 0; i < numThreads; i++){
                    threads[i].join();

                    totalsum += numbers[i].getPartialSum();
                }
            } catch (InterruptedException e) {
                System.out.println(x: "Error");
            }

            System.out.println(x: "==========================");
            System.out.println("Hasil akhir penjumlahan: " + totalsum);
            System.out.println(x: "==========================");

            scanner.close();
    }
}
```

Screenshot Hasil

```
PS C:\Fawwaz\Tugas Kuliah\Tekprom\TekProm(pr)\Java_Week_7>  & '
aceStorage\76670915e580c1b9877abf9ef0754c87\redhat.java\jdt_ws\
Masukkan jumlah thread:
4

Masukkan angka akhir:
1000
 Thread 2 sedang menjumlahkan: 251 sampai 500
 Thread 4 sedang menjumlahkan: 751 sampai 1000
 Thread 1 sedang menjumlahkan: 1 sampai 250
 Thread 3 sedang menjumlahkan: 501 sampai 750
============================
Hasil akhir penjumlahan: 938750
============================
```

Penjelasan

Membagi kasus ke beberapa thread untuk di jalankan kemudian hasil dari setiap thread akan di kalkulasikan menjadi jumlah akhir dari sebuah penumlahan. setiap thread memiliki bobot yang sama tidak boleh beda.

Nama Teman Hal yang Dibantu (Opsional)